Ride-Hailing as an ML System

Scaler School of Technology | ML System Design and LLMOps | Week 8, Session 3, Bonus Material

Bonus material, Scaler School of Technology. This is not part of the course. A trimmed excerpt covering only **deep-dives 1 to 3** (Marketplace Forecasting, Upfront Fare Prediction, Surge Pricing) as worked examples of the rubric. **Practice the remaining sixteen deep-dives on your own** using the ML map and the rubric demonstrated here.

One Tap, Sixty-Three Models

A rider in Bengaluru opens Rapido, types "Indiranagar" and taps **Confirm**. Roughly 400 ms later a bike is assigned, a price shown, an arrival time promised. Dozens of ML models ran in that window — and dozens more in the hours before, to make it possible.

This session maps **all** of them. Not because you will build all of them, but because ride-hailing is the sharpest teaching example of a property the rest of this course has only hinted at: **ML systems that must cooperate, under a shared latency budget, while optimising objectives that actively harm each other**.

Why this domain and not another. A recommender system has one customer. A fraud model has one objective. A ride-hailing marketplace has **two populations whose interests are opposed**, a **physical constraint** (cars cannot teleport), and a **feedback loop** in which today's dispatch decisions determine tomorrow's training data. Every hard idea in ML system design shows up here at once.

Three Properties That Break Your Existing Mental Model

1

The two sides are in direct conflict

Raise surge: drivers earn more and supply rises, but rider conversion falls. Lower it: riders convert, drivers log off. **There is no single metric to maximise.** Every model here is a negotiated settlement between two populations, so "the model improved" invites one question first — *improved for whom?*

2

Your A/B test is statistically invalid

Put 50% of riders in a surge treatment. They book more cars — cars now unavailable to **control** riders, whose experience degrades *because of* the treatment. Control is contaminated. This violates SUTVA, the assumption every t-test you have run depends on. **Randomising users does not work in a marketplace.** Page 9.

3

The system trains on its own decisions

Dispatch sends drivers to the airport. Tomorrow's data shows high airport trip volume, so the demand model learns the airport is high-demand, so dispatch sends more drivers there. A neighbourhood never served records zero demand forever. **You are not observing demand; you are observing your own policy.** Unserved demand is invisible by construction.

The Latency Budget Is the Real Constraint

When the rider taps Confirm the pipeline has roughly **400–600 ms** before the interface feels broken, divided across every model that must run. This constraint kills more elegant designs than accuracy ever will — a model 3% better and 200 ms slower is usually **rejected**.

Stage	Budget	What runs, and why it cannot be slower
Geocode + snap	30 ms	Address → coordinates → road segment. Cached aggressively.
Fare + surge	60 ms	Surge is a lookup of a pre-computed grid, *not* a live model call.
ETA (pickup + trip)	120 ms	Largest slice. Routing engine plus a learned residual.
Fraud screen	50 ms	Parallel with pricing. A hard timeout returns "allow" — fail-open: blocking a legitimate rider costs more than one fraudulent trip.
Dispatch match	150 ms	Assignment over the eligible pool. Scales with pool size, so the pool is pruned first.

Read the fraud row again. Under time pressure the system *lets suspected fraud through*. Not a bug — a deliberate, quantified business decision. Knowing **which way to fail** is a system design skill.

The ML Map — Part 1 of 3

Clusters A–C | Demand, Pricing, ETA | Models 1–22

A reference map. Scan it, mark what you could design today, circle what you could not. **Every model here is designed in full later in this document** — the bracketed number after each cluster heading gives the deep-dive that covers it.

A Demand and Supply — 6 models — deep-dive 1

1. **Spatio-temporal demand forecast.** Rides per hexagon per 15 min. H3/S2 grid; horizons from 5 min to 7 days. Regressors: holidays, weather, events, paydays.
2. **Supply forecast.** How many drivers will be *online* in hex H at time T — a within-session behaviour problem, not a headcount one.
3. **Supply–demand imbalance prediction.** The decision variable feeding surge and repositioning. Forecasting the *ratio* directly beats forecasting both sides separately.
4. **Event and anomaly detection.** A concert ends, rain starts, a metro line fails. Step-change detection that must not fire on ordinary seasonality.
5. **Weather ingestion and effect modelling.** Rain raises demand and lowers supply *simultaneously* — the marketplace worst case.
6. **Long-horizon capacity planning.** Driver recruitment targets per city per month. Feeds finance, not the app.

B Marketplace Pricing — 7 models — deep-dives 2, 3, 4

7. **Surge / dynamic pricing multiplier.** The multiplier per cell per minute.
8. **Price elasticity modelling.** Rider willingness-to-pay. Must be *causal*: observational data says high prices correlate with high demand — backwards as a control signal.
9. **Upfront fare prediction.** Quote a fixed price before the trip: predict the trip's *cost* under asymmetric loss — under-quote and the company absorbs the gap.
10. **Driver incentive optimisation.** Quests, streaks, guarantees. Budget-constrained assignment over uplift — pay only drivers whose behaviour actually changes.
11. **Rider promotion targeting.** Uplift segmentation: persuadables, sure things, lost causes, sleeping dogs. Discounting a sure thing is pure margin loss.
12. **Take-rate / commission optimisation.** The driver/platform split, under two-sided retention constraints and regulatory caps.
13. **Multi-product fare estimation.** Bike, auto, pool, XL, rental — different cost structures and elasticities, cannibalising each other.

C ETA and Routing — 9 models — deep-dives 5, 6, 7

14. **Pickup ETA.** Driver to rider. Short horizon, high variance, the number riders judge most harshly.
15. **Trip ETA.** Pickup to dropoff. Routing-engine baseline plus a learned residual.
16. **Road-segment speed prediction.** Live traffic from your own fleet's GPS pings — your drivers are your traffic sensors.
17. **Map matching.** Snapping noisy GPS to the road graph. HMM + Viterbi, increasingly with learned emissions.
18. **Route choice prediction.** Not the optimal route — the one this driver will *actually* take. ETA predicts reality, not policy.
19. **Pickup point recommendation.** "Where do I meet you." Dominates airports, malls, stadiums; a top cancellation cause elsewhere.
20. **Destination prediction and autocomplete.** Personalised POI ranking. Get the top-3 right and the rider never types.
21. **Geocoding and POI resolution.** Free-text address to coordinates. **The hardest problem for Gojek, Grab and Rapido** — much of SEA and India lacks reliable street addressing.
22. **Arrival detection and geofencing.** Has the driver actually arrived? Gates waiting charges, so it is adversarially probed.

The ML Map — Part 2 of 3

Clusters D–E | Matching, Dispatch, Trust and Safety | Models 23–49

D Matching and Dispatch — 17 models — deep-dives 8, 9, 10, 11

23. **Driver–rider matching.** Bipartite assignment over a batched window.
24. **Learned cost function inside the matcher.** The optimiser is an LP; its *edge weights* are ML predictions. The model is never evaluated alone.
25. **Dispatch as sequential decision-making.** RL and value-function approaches. Myopic matching versus long-run fleet positioning.
26. **Batching window tuning.** Wait 5 s for a better match or assign instantly? Waiting buys match quality and spends rider patience.
27. **Ride-pooling match.** Route compatibility, detour tolerance, inserting a second rider into a live trip.
28. **Driver acceptance prediction.** Will this driver accept? Determines *who to ping first* — a likely-rejector wastes the batching window.
29. **Idle-driver repositioning.** Where to go when empty. Guidance, not instruction — contractors ignore bad advice.
30. **Cancellation prediction.** Both sides, before and after match. Feeds pre-emptive re-matching.
31. **Multi-product supply routing.** One pool serving rides, food and parcels — the Grab/Gojek super-app problem.

The driver-convenience block (32–39). Every item below is a feature drivers *want*, and every one is a **constraint that shrinks the eligible-driver pool** — degrading global matching efficiency. The cleanest illustration of the two-sided tension in this session; quantified in deep-dive 8.

32. **Return-to-home / destination mode.** Driver sets a home location; dispatch offers only rides whose dropoff lies along the homeward corridor. Capped at 2 uses/day to stop cherry-picking.
33. **Homeward-corridor demand forecasting.** Will a homeward-compatible ride even *exist* in the next 20 min, or should the driver deadhead home?
34. **Driver preference and constraint modelling.** Trip-length bands, cash vs digital, AC, vehicle capacity, and female-passenger-only matching — offered by Uber India, Ola and Rapido.
35. **Shift-end / go-offline prediction.** Predict when a driver intends to stop; stop assigning long outbound trips before then.
36. **Break and rest-compliance scheduling.** Rest windows and fatigue limits enter dispatch as *hard* constraints, not penalties.
37. **Fuel and charge-aware dispatch.** EV state-of-charge, real remaining range, charging detours. Central in Delhi and Jakarta fleets.
38. **Driver earnings-goal modelling.** Many drivers work to a daily target then log off, making supply *backward-bending*: raise per-trip pay and some work *fewer* hours.
39. **Heat-map and guidance surfacing.** What the driver sees. Guidance nobody follows has zero effect, however accurate.

E Trust, Safety and Fraud — 10 models — deep-dives 12, 13, 14

40. **Payment fraud detection.** Stolen cards, chargebacks. Extreme imbalance, hard real-time budget.
41. **Incentive and promo abuse.** Fake accounts farming sign-up bonuses; referral rings.
42. **Collusion detection.** Driver and rider faking trips to harvest incentives. A **graph** problem — neither account looks odd alone.
43. **GPS spoofing detection.** Faked location to jump an airport queue or fake a trip that never happened.
44. **Account takeover and device fingerprinting.** Login anomaly plus device-reputation signals.
45. **Driver identity verification.** Periodic selfie matched to the licence photo; catches account sharing and rental.
46. **Crash and harsh-driving detection.** Phone accelerometer and gyroscope telematics. Triggers safety outreach.
47. **Route deviation and in-trip anomaly detection.** Unexpected stop, long halt, route far off plan.
48. **Safety incident triage.** NLP over SOS reports and tickets, ranked by severity. Mis-ranking here is the company's highest-stakes error.
49. **Document verification.** OCR plus forgery detection on licences, insurance, registration.

The ML Map — Part 3 of 3

Clusters F–H | Discovery, Support, Lifecycle | Models 50–63

F Content, Ranking and Discovery — 5 models — deep-dive 15

50. **Home-screen vertical ranking.** Which service — rides, food, payments, parcels — to surface first, now.
51. **Merchant and food recommendation.** GrabFood and GoFood run a full recommender inside the ride-hailing app.
52. **Search and autocomplete ranking.** Merchants and destinations, typo-tolerant across many scripts.
53. **Ads and sponsored placement.** pCTR and pCVR feeding an auction.
54. **Notification send-time optimisation.** When to push and when to stay silent. Fatigue control beats targeting.

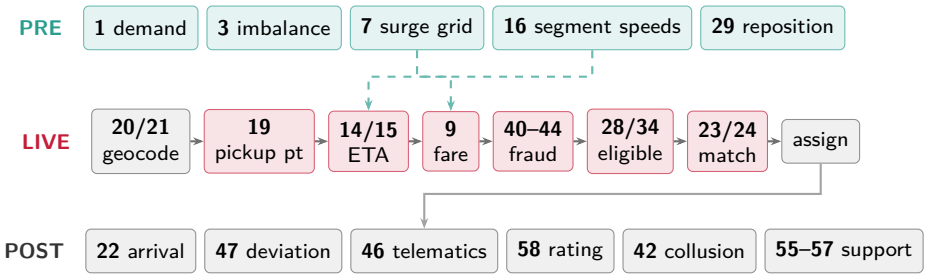
G Customer Experience and Operations — 6 models — deep-dives 16, 17

55. **Support ticket classification and routing.** Intent plus urgency, in a dozen languages.
56. **LLM support agent with tool use.** Trip lookup, refunds, escalation — **the W8S1 architecture**, deployed on ride disputes.
57. **Automated refund adjudication.** "Driver took a longer route — refund?" Replay the GPS trace against the planned route and reason about the gap.
58. **Driver quality scoring.** Ratings are sparse, biased, nearly all 5.0. Recovering a usable signal is itself a modelling problem.
59. **Review and content moderation.** Multilingual abuse detection — Bahasa, Thai, Vietnamese, Hindi, Tamil, and code-mixed Hinglish.
60. **Voice and IVR interfaces.** ASR for drivers with low text literacy and occupied hands. A core requirement at Rapido and Gojek, not an afterthought.

H Lifecycle and Growth — 3 models — deep-dives 11, 18, 19

61. **Driver churn prediction and intervention.** Driver acquisition cost dwarfs rider acquisition cost — the highest-leverage retention model in the company.
62. **Rider lifetime value and churn.** Feeds promotion budgets and win-back campaigns.
63. **Geo-experimentation and switchback testing. Not a model — the measurement infrastructure every other model depends on.** Page 9.

Trace One Ride Request: What Actually Fires, and When



Read the top row again. Surge (7) and segment speeds (16) are **not called** during the request — they are pre-computed on a schedule and read from a key-value store as a lookup. This is the most common way real systems buy latency: **move the model out of the request path and turn its output into a cache**. When your design has too many models in a 400 ms budget, the question is never "which model do we cut?" — it is "which of these can be pre-computed?"

Nineteen Systems, Sixty-Three Models

How the catalogue maps onto the designs that follow

Why nineteen designs and not sixty-three. No company staffs a team per model. **Models that share data, infrastructure and a consumer are built as one system** — the six demand and supply forecasts are one forecasting platform, not six projects; the four geocoding and pickup models are one location service. **Grouping the way industry actually groups is itself part of the system design skill**, and it is what stops a design answer from becoming a list. Each of the nineteen below is worked through the full 10-step rubric.

#	System	Models	The idea it teaches
1	Marketplace Forecasting	1–6	Censored demand; you observe your own supply
2	Upfront Fare Prediction	9, 13	Quantile regression; bias, not variance, bankrupts you
3	Surge Pricing	7	Causal elasticity; why logged price is confounded
4	Incentives and Promotions	8, 10–12	Uplift, not propensity; the knapsack allocator
5	Location Intelligence	19–22	The emerging-market core problem
6	Traffic and Map Inference	16–18	Your fleet is your sensor network
7	ETA Prediction	14, 15	Residual learning; asymmetric loss
8	Dispatch and Matching	23–32, 34	ML inside an optimiser; constraints cost efficiency
9	Ride Pooling	27, 31	Dynamic insertion; the first rider pays
10	Driver Positioning	25, 29, 33, 39	Herding; advice nobody must follow
11	Driver Lifecycle	35, 36, 38, 61	Multi-homing; backward-bending supply
12	Payment Fraud and Abuse	40–42	Extreme imbalance; graph signal; adversaries
13	In-Trip Safety	43, 46–48	A different cost function entirely
14	Identity and Documents	44, 45, 49	False positives take away income
15	Super-App Discovery	50–54	Cross-vertical cannibalisation of supply
16	Support and Disputes	55–58	Adjudication with an absent second party
17	Multilingual and Voice	52, 59, 60	Unequal model quality is unequal service
18	Rider Lifecycle	26, 30, 62	Cancellation as a churn event
19	Experimentation Platform	63	The infrastructure all of the above depend on

Reading This Document

Each deep-dive follows the same ten steps, so you can read across them to see how the *same* framework produces very different designs:

Steps	What to look for
1–3	**Clarify, context, problem statement.** Notice how often the clarifying question — "requests or completed trips?", "who bears the risk?", "who is the absent party?" — determines the whole design.
4	**Requirements**, split functional and non-functional.
5	**Estimations. Read these carefully.** In nearly every system the arithmetic *selects* the architecture: 2×10^8 matching pairs against a 150 ms budget forces candidate pruning; 1.2 MB of surge state forces a lookup; 300k forecast series forbids per-series models; review capacity sets the fraud threshold.
6	**Architecture**, decomposed into the six sub-layers (a)–(f): data collection, data engineering, feature engineering, preprocessing, models, post-processing.
7	**Evaluation**, offline and online — and repeatedly, why the offline metric misleads.
8	**A/B testing. Track which systems can use ordinary randomisation and which cannot.** It is roughly half and half, and the reason is always shared scarce supply.
9	**Deployment and monitoring**, including the fallback ladder and which way the component fails.
10	**Risk and trade-offs. This is where the judgement lives**, and it is what an interview is actually assessing.

If you read only three. For the sharpest single ideas: **deep-dive 3** (why logged data gives elasticity the wrong sign), **deep-dive 8** (ML as coefficients inside an optimiser, and what return-to-home costs), and **deep-dive 19** (why a marketplace cannot use the A/B test you already know). Those three generalise furthest beyond ride-hailing.

Deep-Dive 1 — Marketplace Forecasting

Models 1–6 | Demand, supply, imbalance, events, weather, capacity

Why these six are one system, not six projects. No team builds a "demand model" and a separate "supply model" with separate pipelines. They share the same spatial grid, the same time buckets, the same event and weather regressors, and the same backfill machinery — and critically, **the thing everything downstream consumes is neither forecast alone but their ratio**. Building them separately produces two models with inconsistent geometry that cannot be divided.

Step 1 | Clarify the Problem

Questions to ask: What consumes the forecast — surge, repositioning guidance, driver incentives, or quarterly hiring? (Each needs a different horizon and a different error tolerance.) What spatial resolution? What is the cost of over-forecasting versus under-forecasting supply? Do we forecast *requests* or *completed trips*?

The clarification that determines everything: requests or completed trips? Your logs record completed trips. But completed trips are $\min(\text{demand}, \text{supply})$ — **censored by the supply you happened to have**. In an under-supplied cell, completed trips tell you how many drivers were there, not how many riders wanted one. Train on completed trips and you build a model that forecasts your own historical supply, then use it to allocate supply, and the shortage becomes permanent and invisible. **You must forecast true demand, which means recovering it from censored observations** — app opens, searches, requests that timed out, and requests abandoned after seeing the ETA.

Step 2 & 3 | Context and Problem Statement

Context. Forecasting is upstream of nearly everything. Surge reads the imbalance; repositioning guidance reads the forecast gradient; incentive budgets are allocated days ahead from it; driver recruitment is planned quarterly from it. **A forecasting error does not surface as a forecasting complaint** — it surfaces as bad pricing, drivers sent to the wrong place, and wasted incentive spend.

Problem statement. For each cell h in an H3 grid and each future time bucket t, produce (i) expected ride *requests* $\widehat{D}_{h,t}$ corrected for historical censoring, (ii) expected available driver-hours $\widehat{S}_{h,t}$, and (iii) the predictive distribution of their ratio — over horizons from 5 minutes to 90 days, refreshed every 5 minutes for short horizons and daily for long ones.

Step 4 | Define Requirements

Functional	What the system must do
Multi-horizon	One coherent system from 5 min (dispatch, surge) to 90 days (hiring). Forecasts must not contradict each other across horizons.
Multi-resolution	Cell, zone, and city level, **reconciled** so cell forecasts sum to the city forecast.
Censoring correction	Recover latent demand from supply-constrained observations.
Distributional output	Surge and incentives need uncertainty, not a point estimate. A cell with high variance should be treated differently from a confidently-busy one.
Event awareness	Ingest a calendar of concerts, matches, festivals, flight schedules.
Backfill	Recompute history when the road graph, grid, or a definition changes.
Non-functional	**Target**
Freshness	Short-horizon forecast no older than 5 min.
Serving	A lookup, never a live model call.
Scale	~150k active cells × multiple horizons.
Graceful degradation	Fall back to a seasonal-naive baseline (same cell, same hour, last week) that is always available.

Step 5 | Estimations

- **Series count:** 150k cells × 2 quantities (demand, supply) = **300k time series**. At 15-minute buckets that is 96 points/day/series, or ~29M new observations per day.
- **This rules out per-series models immediately.** 300k individually-fitted ARIMA or Prophet models is operationally absurd — you cannot monitor, retrain or debug 300k artefacts. **The estimate forces a single global model** with cell identity as a feature (typically a learned embedding), which is also how modern forecasting practice has converged.
- **Sparsity:** the majority of cells see fewer than 5 rides per bucket. A large fraction of buckets are **zero**. This is count data with excess zeros — not a Gaussian regression problem, which is the most common modelling mistake here.
- **Training data:** 2 years × 29M/day ≈ 21B rows. Aggregate and sample; do not attempt to train on all of it.

- **Storage for served forecasts:** 300k series × 20 horizons × 3 quantiles × 4 bytes ≈ **72 MB**. Trivially cacheable in memory at every edge — the same pattern as surge state.

Forecasting — Step 6: System Architecture

a

Data Collection

The demand-side signal chain matters more than the trip table. Log app opens, destination searches, fare-quote views, requests issued, requests that found no driver, and requests abandoned after seeing the price or ETA. **Each is a progressively stronger signal of intent, and the drop-offs between them are what let you reconstruct unmet demand.** Supply side: driver online/offline events, position pings, and status (idle, en route, on trip). Plus the event calendar, weather feed, and a holiday calendar that must be regional — India's holidays vary by state and materially change travel.

b

Data Engineering

A streaming path aggregates events into H3 cell × 5-minute buckets for the short-horizon model; a batch path builds the training table with **point-in-time correctness**. The classic bug: joining today's weather to a historical row using the *observed* weather rather than the *forecast* that was available at the time. **Train on the forecast you would have had, not the truth you learned later**, or the model leans on information it will never have in production.

c

Feature Engineering

Temporal — hour-of-week encoded cyclically (sine/cosine, so 23:00 and 00:00 are adjacent), holiday flags, days-to-payday (strong in cash economies). **Lags** — same cell at $t-1$, $t-96$ (yesterday), $t-672$ (last week); rolling means over multiple windows. **Spatial** — the same lags for neighbouring cells in the H3 k-ring, because demand moves between adjacent cells. **Static cell** — land use, POI density, transit proximity, historical mean and variance. **Exogenous** — weather forecast, event proximity in space and time, flight arrivals for airport cells.

d

Preprocessing

Censoring correction is the hard part: for buckets where supply was clearly binding (near-zero idle drivers, high request timeout rate), the observed count is a lower bound on demand. Options: model only uncensored buckets and extrapolate; use a Tobit-style censored likelihood; or use the funnel (searches → requests) to estimate the conversion that *would* have occurred. **Also:** aggregate sparse cells upward rather than modelling near-empty series, and flag anomalous historical periods (a city-wide outage, a lockdown) so they do not poison the seasonal profile.

e

Models

Baseline that is hard to beat: seasonal-naive — same cell, same hour, same day-of-week, last week. Report every model against it; many production forecasting systems beat it by less than their owners assume. **Production workhorse:** a single global gradient-boosted model over all cells, with cell identity as a categorical and the lag features above. Cheap, robust, debuggable. **Where neural helps:** a global deep model (temporal-convolution or transformer style) that learns cell embeddings and shares structure across cells — genuinely useful for the many sparse cells that lack data to learn alone. **Count-aware loss:** use a Poisson or negative-binomial objective, or quantile loss for the distributional output. Squared error on sparse counts systematically over-predicts quiet cells.

f

Post-processing

Hierarchical reconciliation — raw cell forecasts will not sum to the zone or city forecast. Reconcile (MinT or a simpler proportional method) so downstream consumers see a consistent picture; an incentive budget allocated from unreconciled cell forecasts will over- or under-spend the city budget. Clip negatives, apply event overrides where a known large event dominates, and write to the forecast store.

Step 7 | Evaluation: Offline and Online

Metric	Why, and the trap
WAPE (weighted abs % error)	Preferred over MAPE for count data — **MAPE is undefined or explosive when actuals are zero**, which is most of your cells.
Pinball loss	The right metric when you output quantiles. Evaluates the whole distribution, not just the median.
Bias by cell and hour	**Systematic** over- or under-forecast is far worse than noise, because downstream allocation compounds it in one direction.
Skill vs seasonal-naive	The only honest framing. "WAPE 18%" is meaningless; "12% better than seasonal-naive" is a result.
Accuracy at the peaks	Average error is dominated by quiet buckets nobody cares about. **Evaluate on the top-decile-demand buckets separately** — those are the ones the business acts on.

Online: the forecast is not consumed directly by users, so measure it through its consumers — did surge over-trigger, did repositioning guidance send drivers to cells that stayed quiet, did incentive spend land where demand materialised? **Track "guidance regret": drivers who followed the forecast and earned less than those who ignored it.**

Step 8 | A/B Testing

You cannot A/B test a forecast directly — everyone shares one view of the future. You test **the decisions it drives**, which inherits the interference problem: a repositioning or surge change requires a **switchback**. What you *can* do cheaply is **backtesting with rolling origins** — replay history, forecast forward from each origin, compare. This is valid because a forecast, unlike a policy, does not change the world it is predicting. **Guard against leakage:** the rolling-origin evaluation must only use features available at the origin.

Step 9 | Deployment and Monitoring

- **Forecasts are written, not served.** A scheduled job computes and publishes to a store; the request path reads. Same pattern as surge.
- **Monitor:** forecast-vs-actual as actuals arrive (an automatic, delayed label), bias drift per city, feature freshness, the fraction of cells falling back to the naive baseline, and **reconciliation residuals**.
- **Alert on structural breaks** — a new metro line, a road closure, or a competitor's promotion changes the data-generating process, and the model will keep confidently forecasting the old world for weeks.
- **Retrain weekly**; refit seasonal components less often to avoid chasing noise.

Step 10 | Risk Management and Trade-offs

The self-fulfilling forecast

Forecast low demand in a cell → send no drivers → riders there fail to get cars and stop opening the app → observed demand falls → the forecast is "confirmed". **The model is not predicting the world; it is creating it.** Neighbourhoods can be quietly written out of service this way, and the metric dashboard will show excellent forecast accuracy throughout. **Mitigations:** forecast *requests and app-opens*, not completed trips; maintain deliberate supply coverage in low-forecast areas; monitor unmet-demand and app-open-without-request rates per cell as a first-class equity metric.

Black-swan periods poison the training window

A pandemic, a lockdown, a month-long transit strike. Include them and the seasonal profile is wrong; exclude them and you cannot forecast the next one. **Practical answer:** flag anomalous periods explicitly as features rather than silently dropping them, keep a human override path, and accept that no model handles a genuine regime change — the system requirement is that it **fails visibly** rather than confidently.

Accuracy versus actionability

A forecast accurate to ± 2 rides in a cell expecting 3 is useless for decisions; a forecast accurate to ± 50 in a cell expecting 800 is highly actionable. **Chasing aggregate WAPE pushes effort toward the many sparse cells where accuracy does not change any decision.** Weight your evaluation — and your engineering attention — by the decisions the forecast actually drives.

Deep-Dive 2 — Upfront Fare Prediction

Models 9 & 13 | Quoting a fixed price before the trip exists

Why fare prediction is not the same as ETA. ETA predicts time. Upfront fare commits the company to a **price** based on predicted time and distance — and once quoted, **the company absorbs the difference**. The moment you show a fixed price, every ETA error becomes a P&L entry. This converts a prediction problem into a **risk-pricing problem**, and it is why fare and ETA are built by the same team but with different loss functions.

Step 1 | Clarify the Problem

Questions: Is the quote binding, or an estimate that reconciles to metered actuals? Who absorbs the gap — platform, driver, or rider? Is the driver paid on quoted or actual time? (This single answer determines whether the driver has an incentive to take a longer route.) What happens on a mid-trip destination change? Do we quote per product, and can products cannibalise?

The critical design question: who bears the risk? If the rider pays the quote and the driver is paid on actuals, the **platform** absorbs variance — and needs an accurate, unbiased model plus a reserve. If the driver is paid the quote-derived amount, **the driver** absorbs it, which is cheaper for the platform and deeply unpopular, because drivers stuck in unexpected traffic earn less per hour through no fault of their own. **Most platforms keep the risk centrally, precisely because the platform can pool it across millions of trips while an individual driver cannot.** That pooling argument is the answer an interviewer is looking for.

Step 2 & 3 | Context and Problem Statement

Context. Upfront pricing exists because metered fares create anxiety — riders do not know what they will pay, which suppresses booking. A fixed price raises conversion measurably. **The platform is selling certainty and charging for it**, and the cost of providing that certainty is the variance of its own prediction errors.

Problem statement. Given origin, destination, product, time and current conditions, output a price such that the **expected** realised cost is covered, the quote is competitive enough to convert, and the **aggregate** quoted-minus-actual gap across all trips is a controlled, budgeted number rather than an uncontrolled leak — computed within the 60 ms pricing slice.

Step 4 | Define Requirements

Functional	**Detail**
Multi-product quoting	Bike, auto, car, XL, pool, rental — shown simultaneously and mutually consistent (an XL must not be cheaper than a bike).
Cost-side prediction	Predict duration and distance distributions, not point values.
Risk margin	An explicit, tunable buffer, separate from the point prediction.
Re-quote rules	Defined behaviour for destination change, long waits, extra stops.
Price consistency	The same trip quoted twice within seconds must not differ visibly.
Non-functional	**Target**
Latency	$<$60 ms for all products together.
Aggregate bias	$\lvert$quoted $-$ actual$\rvert$ mean within a stated tolerance of zero, monitored daily.
Auditability	Every quote reconstructible for dispute resolution.

Step 5 | Estimations

- **Quote volume:** browse traffic dominates. At $4\times$ the booking rate and 6 products quoted per view, 3M bookings/day $\Rightarrow$ ~72M product-quotes/day, ~5k QPS peak. **Quoting is far higher volume than booking** — design for the browse path, not the purchase path.
- **The economics of being wrong.** Suppose mean absolute quote error is $\mathrm{Rs.}\,8$ on a $\mathrm{Rs.}\,150$ fare, symmetric. Symmetric error is *not* the problem — it nets out. **A 2% systematic under-quote is:** 3M trips $\times\ \mathrm{Rs.}\,3 = \mathbf{Rs.\ 9M/day} \approx \mathrm{Rs.}\,3.3\mathrm{B/year}$. **Bias, not variance, is what bankrupts you**, which is why the requirement above is written on aggregate bias.
- **Tail exposure:** 1% of trips take $> 2\times$ the predicted time. If those cost $\mathrm{Rs.}\,200$ extra each, that is 30k $\times\ \mathrm{Rs.}\,200 = \mathrm{Rs.}\,6\mathrm{M/day}$ of tail loss — a real reserve line, and the reason the model must output a distribution.

Fare — Step 6: System Architecture

The fare is a **composition**, not a single model output:

$$\text{Fare} = \underbrace{\text{base}}_{\text{product}} + \underbrace{r_d \cdot \widehat{d}}_{\text{distance}} + \underbrace{r_t \cdot \widehat{T}}_{\text{duration}} \times \underbrace{m}_{\text{surge lookup}} + \underbrace{\rho(\sigma)}_{\text{risk margin}} + \text{tolls, fees}$$

a

Data Collection

Completed trips with **actual** duration, distance, waiting time and tolls; the quote that was shown; whether the rider converted at that quote (**the elasticity signal**); and the reconciliation outcome. Log **quotes that were not accepted** — they are the only evidence of price sensitivity and are routinely dropped from logging to save volume.

b

Data Engineering

Shares the ETA feature pipeline entirely — segment speeds, route features, temporal features. **Do not build a parallel pipeline**; build a second consumer of the same features with a different head and loss. A separate pipeline guarantees the fare and the displayed ETA eventually disagree, which riders notice immediately.

c

Feature Engineering

Everything ETA uses, plus: **toll-road probability** on the likely route, **waiting-time expectation** (traffic-light density, pickup-point difficulty), **destination ambiguity** — a vaguely specified destination has higher variance and deserves a larger margin — and **product cross-features**, since a bike and a car on the same route have different duration distributions.

d

Preprocessing

Exclude trips with mid-trip destination changes from the base model and handle them via the re-quote rule instead. **Critically: do not exclude the long tail.** It is tempting to winsorise extreme durations as you do for ETA, but for fare **the tail is the financial exposure** — if you train only on trimmed data, your risk margin is calibrated to a world without bad days.

e

Models

Quantile regression rather than mean regression: predict the 50th, 80th and 95th percentiles of duration. Gradient-boosted trees with a pinball loss do this well and cheaply. The median feeds the point estimate; the spread between quantiles **is** the uncertainty σ that sets the risk margin. **This is the elegant part of the design:** one model gives you both the price and the confidence to charge for, with no separate uncertainty model.

f

Post-processing

Apply the surge multiplier as a lookup (never recompute it here). Add the margin $\rho(\sigma)$, larger where uncertainty is larger. **Enforce cross-product monotonicity** — sort products by cost tier and correct violations, because an XL quoted below a bike is an immediate exploit. Round to rider-legible values. Cache the quote briefly so a refresh does not show a different number.

Step 7 | Evaluation: Offline and Online

Metric	Why
Aggregate quote bias	**The number that matters.** Mean(quoted − actual) across all trips, tracked daily per city. Must sit within a stated band around zero.
Pinball loss by quantile	Are the quantiles honest? If the 95th percentile is exceeded 12% of the time, the risk margin is systematically too thin.
Tail loss (p99 gap)	Your reserve requirement.
Conversion at quote	The demand side — an accurate but uncompetitive quote loses the trip.
Per-segment bias	Slice by product, distance band, hour, city. **Aggregate bias can be zero while systematically losing money on bikes and making it back on cars.**

Online: realised margin per trip, dispute and refund rate for fare complaints, conversion by product, and the frequency of re-quote events.

Step 8 | A/B Testing

Rider-level randomisation is **mostly** valid — quoting one rider differently does not consume shared supply. **But two cautions:** (i) if the treatment quotes systematically lower, treated riders book more and *do* draw supply, reintroducing interference at large traffic shares; (ii) price tests raise fairness concerns — **charging different people different prices for the same trip is legally and reputationally sensitive**, so randomise the *model*, not personalised prices, and document that the variation is a bounded experiment on estimation accuracy rather than on willingness-to-pay. **Guardrails:** conversion, realised margin, dispute rate.

Step 9 | Deployment and Monitoring

- **Shadow-quote** for a week and compare against metered actuals before any rider sees the new model.
- **Daily bias reconciliation** is the core operational ritual: sum quoted minus actual by city and product; alert if it drifts outside the band. **This is a finance-grade control, not just an ML dashboard.**
- **Fallback:** model unavailable → fall back to metered pricing and tell the rider the fare is estimated. **Never guess a binding price without a model** — an un-modelled fixed quote is an open-ended liability.
- **Circuit breaker** on aggregate bias: if the day's running gap exceeds a threshold, revert to the previous model version automatically.

Step 10 | Risk Management and Trade-offs

!

Adverse selection on destination ambiguity

Riders with vague destinations ("somewhere near the market") generate high-variance trips. If your margin does not scale with uncertainty, these trips are systematically underpriced — and riders who habitually enter vague destinations are systematically subsidised by everyone else. **This is adverse selection, and it grows over time** as the subsidised behaviour is rewarded. The $\rho(\sigma)$ term exists specifically to price it.

!

Driver incentive misalignment

If the driver is paid from the quote and the quote is based on predicted time, a driver who takes a *longer* route earns the same for more work — so they optimise for speed, which is good. But if the driver is paid on *actual* time while the rider pays a fixed quote, the driver now benefits from a slower trip, and the platform pays for it. **Check the incentive direction of every pricing rule you write**; a pricing scheme that pays people to waste time will reliably produce wasted time.

!

Certainty versus fairness in the tail

Fixed pricing pools risk: on a bad-traffic day the rider is protected and the platform absorbs it. That is the product. **But it means riders on unusually fast trips overpay relative to cost** — they are funding the pool. This is insurance and it is defensible, but it must be explained honestly, and the margin must be a stated, bounded number rather than a quietly growing one. **An opaque, creeping risk margin is how a pricing system loses regulatory goodwill.**

Deep-Dive 3 — Surge Pricing

Model 7 | Causal, not correlational | The model you cannot A/B test normally

Step 1 | Clarify the Problem

Surge is the most misunderstood model in ride-hailing, including by the people who build it. Start by rejecting the common framing.

Surge is not a revenue-extraction model. If it were, you would simply raise prices everywhere and keep the money. Surge exists to solve an **allocation** problem: when 100 riders want 40 cars, *someone* is not getting a car. The only questions are who, and by what mechanism. Without price, allocation happens by queue — everyone waits, nobody knows how long, and drivers have no reason to move toward the shortage. With price, demand is rationed to those who value the trip most *right now*, and supply is actively pulled toward the shortage. **The output is not a price. The output is a reallocation of cars across the city.** If you frame surge as pricing, you will build the wrong metrics.

Questions to ask: Is there a regulatory cap in this market? (India has had 1.5–2× caps; Indonesia mandates fare floors *and* ceilings.) Is the price shown before booking or after? Do we surge the rider price, the driver payout, or both — and are they allowed to differ? What is the smallest geographic unit we can price at? How often can the multiplier change without the experience feeling erratic?

Step 2 & 3 | Context and Problem Statement

Context. Surge is the most politically and reputationally sensitive system in the company. It generates regulatory scrutiny, news cycles during emergencies, and genuine rider anger. It is also the primary short-term lever for marketplace balance — demand forecasting tells you a shortage is coming; surge is one of only two tools (the other is repositioning guidance) that can do anything about it in the next ten minutes.

Problem statement. For each geographic cell and each minute, choose a price multiplier $m \in [1.0, m_{\max}]$ that maximises a weighted combination of completed trips, rider surplus and driver earnings, subject to regulatory caps and a constraint on multiplier volatility — given a forecast of demand, a forecast of supply, and an estimate of **how each side responds to price**. That last term is the entire difficulty.

Step 4 | Define Requirements

Functional	What the system must do
Per-cell pricing	Compute a multiplier per H3 cell per minute across every active city.
Spatial smoothing	Adjacent cells must not differ wildly — a rider walking 50 m to save 40% is a broken experience and is gamed within days.
Temporal smoothing	Rate-limit changes. A multiplier oscillating 1.0/2.0/1.0 minute-to-minute destroys trust on both sides.
Regulatory caps	Enforce per-market ceilings as a hard constraint, auditable after the fact.
Emergency suppression	Automatically cap or disable surge during disasters and emergencies. **Non-negotiable**, and it must not depend on a human noticing.
Rider transparency	Communicate the multiplier before booking, not after.
Non-functional	**Target**
Latency	$<$10 ms — because it is a **lookup**, not a model call. See Step 6.
Update cadence	Recompute every 30–60 s per cell.
Auditability	Every multiplier reconstructible months later for regulatory response.
Blast radius	A bad deploy must not be able to price an entire country at $5\times$.

Step 5 | Estimations

- **Cells:** a mid-size Indian city covers roughly 1,500 H3 resolution-8 cells with meaningful activity ($\sim$0.7 km^2 each). Across 100 cities, $\sim$150k active cells.
- **Compute:** 150k cells recomputed every 30 s = **5,000 multiplier computations/sec**. Trivially small — *this is the key insight driving the architecture*. Surge is a small, cheap, batch computation, and its output is a tiny table of (cell, multiplier) pairs.
- **Storage:** 150k cells × 8 bytes $\approx$ **1.2 MB of hot state.** The entire live surge state of a global ride-hailing company fits in a CPU cache. It is replicated to every edge region and read as an in-memory lookup.
- **Audit log:** 150k cells × 2,880 updates/day × $\sim$200 bytes = $\sim$86 GB/day of pricing audit trail. **The logging is three orders of magnitude more expensive than the computation** — and it is mandatory.

Notice what the estimation just proved. A junior design puts a "surge model" in the request path. The numbers say it should never be there: 5k computations/sec of background work producing 1.2 MB of state that the request path reads in microseconds. **Estimation is not arithmetic ritual — it is what selects the architecture.** This is exactly the "move the model out of the request path" pattern flagged on page 3 of the ML map.

Surge — Step 6: System Architecture

The Mistake Everyone Makes First

The obvious design sets the multiplier from the imbalance ratio:

$$m = \text{clip}\left(\left(\frac{\widehat{D}}{\widehat{S}}\right)^{\alpha},\ 1.0,\ m_{\max}\right)$$

This is a real baseline and it is what most companies ship first. It is also **wrong in an instructive way**: it assumes a fixed relationship between imbalance and the correct price, when the actual quantity you need is *how much demand falls and how much supply rises per unit of price* — and that differs by city, by hour, by cell, and by rider segment. A $1.5\times$ multiplier that clears the market at an airport at midnight may do nothing at all in a business district at 6 pm, where riders are expensing the fare and barely notice the price.

Why you cannot learn elasticity from historical data. Your logs say: when the multiplier was high, demand was high. A naive model concludes *raising prices increases demand*. The causality runs the other way — demand was high, which is *why* the algorithm raised the price. Price was assigned by a policy that looked at demand, so price and demand are confounded by the very thing you are trying to measure. **Fitting a supervised model to logged price and quantity gives you an elasticity estimate with the wrong sign.** This is the single most important idea on this page, and it generalises far beyond surge: **any time your feature was set by a policy that also saw the outcome, supervised learning gives you a correlation you must not act on.**

Getting a Causal Elasticity Estimate

Approach	How it works	Cost
Deliberate randomisation	Add a small random jitter to the multiplier in randomly chosen cells and times. Price now varies for reasons unrelated to demand, so the confound is broken.	Gold standard. Costs real revenue and must be small and bounded.
Switchback experiments	Flip the whole cell between pricing policies on a time schedule. See deep-dive 19.	The workhorse. Needs careful window design.
Natural experiments	Exploit rollout boundaries, regulatory changes, or cap-induced discontinuities.	Free, but rare and hard to defend.
Regression discontinuity	Compare cells just above and just below a rounding threshold in the multiplier.	Clever, cheap, but local to the threshold.

Practical answer: maintain a permanent, small randomisation budget. Roughly 1–2% of cell-minutes get a jittered multiplier, forever. This is a standing cost the company pays to keep its elasticity estimates honest — and teams that skip it discover their pricing model has been quietly mis-calibrated for a year.

Step 6 Decomposition

a

Data Collection

Request events (**including requests that did not convert** — these are the elasticity signal and are easy to forget to log), driver online/offline transitions, driver position, completed trips, the multiplier actually shown, and whether the rider saw and abandoned. **Log the counterfactual context:** what multiplier *would* have been shown under the baseline policy.

b

Data Engineering

Streaming aggregation per H3 cell per minute: open requests, available drivers, in-progress trips, recent conversion rate. Written to the hot state store. A parallel warehouse path retains everything for elasticity estimation and regulatory audit.

c

Feature Engineering

Imbalance — forecast demand, forecast supply, their ratio, and the trend in each. **Elasticity priors** — the estimated demand and supply response for this cell/hour cluster. **Context** — weather, event flags, day type, whether a flight bank just landed. **Neighbourhood** — imbalance in adjacent cells, because supply flows between them and a shortage next door is about to become your shortage.

d **Preprocessing**

Cluster cells into elasticity groups — there is not enough data per cell to estimate a reliable elasticity individually, so pool across similar cells (by density, land use, and typical trip purpose) and use a hierarchical estimate that shrinks a sparse cell toward its group mean.

e **Models**

Three components, not one. **(i)** Demand forecast $\widehat{D}$ and supply forecast $\widehat{S}$ — ordinary supervised time-series models. **(ii)** Elasticity estimates — causal, from the randomised budget above. **(iii)** An **optimiser**, not a model, that chooses m to maximise the objective given (i) and (ii) under the caps and smoothing constraints. **The ML produces inputs; a constrained optimisation produces the decision.**

f **Post-processing**

Spatial smoothing across adjacent cells; temporal rate-limiting; hard clipping at the regulatory cap; emergency override; rounding to rider-legible values. **Then write to the hot store** — the request path only ever reads.

The Supply Side Is Not Symmetric — and It Bends Backwards

Demand response to price is well-behaved: raise the price, fewer people book. **Supply response is not.** Three effects make it strange, and a model that assumes a simple upward-sloping supply curve will mis-price systematically.

- **It is slow and physical.** A driver responding to surge must *drive* to the surging cell. That takes 5–15 minutes, by which time the shortage may have resolved — and now there is a *surplus* of drivers who arrived to find the multiplier gone and their trip unpaid for. **Surge that reacts too fast creates driver resentment and oscillation.**
- **It is backward-bending** (model 38). Many drivers, especially in India and South-East Asia, work toward a *daily earnings target*. Raise per-trip pay and they hit the target sooner and **log off earlier**. Above a certain multiplier, raising the price can *reduce* hours of supply. A monotonic supply-response assumption gets this exactly backwards.
- **It is anticipatory and gameable.** Drivers learn the surge patterns. Some will collectively go offline to manufacture a shortage and trigger a multiplier. Your pricing model is **playing a repeated game against a strategic counterparty that can observe its outputs**, which is a fundamentally different problem from prediction.

Surge — Steps 7 to 10: Evaluation, Testing, Deployment, Risk

Step 7 | Evaluation: Offline and Online

Offline evaluation of a pricing policy is genuinely hard, and you should say so out loud in an interview. You cannot replay history, because under a different price a different set of trips would have happened — the counterfactual is unobservable. What you can do:

- **Forecast accuracy** of the demand and supply components, evaluated normally.
- **Elasticity estimate quality** — confidence intervals from the randomisation budget; are they tight enough to act on?
- **Simulation.** Build an agent-based marketplace simulator, replay a day, and compare policies. **Treat the output as directional only** — the simulator contains your assumptions about behaviour, so a policy that wins in simulation has proved it beats your beliefs, not reality.
- **Constraint satisfaction** — did the policy ever breach a cap, oscillate beyond the rate limit, or produce a spatial cliff between adjacent cells? These are checkable offline and are the ones that produce news stories.

Online metric	Whose interest it represents
Completed trips	**North star.** The marketplace exists to complete trips. Revenue per trip rising while trips fall is a failure.
Request fulfilment rate	Rider side — of people who wanted a car, how many got one?
Rider surplus proxy	Rider side — conversion rate at each price point; abandonment after seeing the multiplier.
Driver earnings/hour	Driver side — the number that determines whether drivers stay.
Driver utilisation	Driver side — fraction of online time that is paid. Surge that summons drivers to a resolved shortage destroys this.
ETA to pickup	Both — the physical outcome surge is supposed to improve.
Surge incidence	Guardrail — % of requests seeing $m > 1$. If this rises steadily, the system has become a revenue tool and regulators will notice.

You must report both sides, always. A surge change that raises driver earnings 8% and cuts completed trips 5% is not obviously good or bad — it is a **decision for the business**, not for the model owner. Presenting only the metric that improved is the most common way ML teams mislead their own leadership, usually without intending to.

Step 8 | A/B Testing — Where the Standard Method Breaks

This is the methodological heart of the session.

×

Why rider-level randomisation is invalid

Assign 50% of riders to a lower multiplier. They book more. **They take the cars.** Control riders now face fewer available drivers, longer ETAs, and worse service — caused entirely by the treatment. The control group is no longer a valid counterfactual for "what would have happened without the change"; it is a measurement of "what happens to people competing against a treated group". The measured effect is inflated, often severely, because you are capturing both the real treatment effect and the harm done to control. **Ship this and the effect vanishes at 100% rollout** — there is no untreated group left to take supply from.

The formal statement: user-level randomisation requires **SUTVA**, the assumption that one unit's treatment does not affect another unit's outcome. A marketplace with shared, scarce supply violates it by construction. The fix is to **randomise at a unit that contains the interference**.

✓ **Switchback randomisation — the standard fix**

Take one city (or one cell cluster) and switch the *entire* unit between policy A and policy B on a repeating time schedule — say every 30 or 60 minutes. Everyone in the region experiences the same policy at the same time, so there is no within-period contamination. The unit of analysis is the **time period**, not the user. **Design considerations:** periods must be long enough that carryover from the previous policy has decayed (drivers repositioned under policy A are still there for the first minutes of policy B), but short enough to accumulate periods for power. Randomise the order rather than alternating strictly, or a weekly pattern will align with your schedule and confound everything. Analysis must cluster standard errors by period — treating individual trips as independent observations inside a switchback grossly overstates significance, and this is the most common analysis error.

✓ **Geo / cluster randomisation — the alternative**

Assign whole cities, or well-separated cell clusters, to treatment or control. Cleaner to analyse, and it handles long carryover effects that defeat switchbacks. **But:** cities are few and heterogeneous, so power is poor and you often need a difference-in-differences or synthetic-control analysis rather than a simple mean comparison. Adjacent regions also leak — a driver can cross a boundary — so clusters need buffer zones.

Carry this beyond ride-hailing. Interference appears in *any* system with a shared constrained resource: ad auctions (bidders compete for the same slot), marketplaces (buyers compete for stock), social feeds (creators compete for the same attention), and two-sided delivery platforms. **The question "do my treatment and control units compete for anything?" should be asked before every experiment you design for the rest of your career.**

Step 9 | Deployment and Monitoring

- **Staged rollout by city**, never globally. Pricing bugs are extraordinarily expensive and highly visible.
- **Hard circuit breakers** that are *independent of the model*: a ceiling that cannot be exceeded regardless of what the optimiser outputs; an automatic kill-switch if the fraction of surging cells exceeds a threshold; an alert if any cell's multiplier changes by more than a set delta in one step.
- **Emergency detection** — automatically suppress surge when a disaster signal fires. Build it before you need it; the failure mode is a national news story.
- **Monitor:** multiplier distribution per city, surge incidence trend, conversion at each multiplier band, driver repositioning response, cap-breach attempts (the optimiser wanting to exceed the cap tells you demand is genuinely unmet), and spatial-gradient violations between neighbouring cells.
- **Audit:** every multiplier reconstructible with its inputs. Regulators do ask, and "the model decided" is not an acceptable answer.

Step 10 | Risk Management and Trade-offs

! **Surge during emergencies is an ethics problem, not a modelling problem**

During a flood, a terror incident or a transit failure, demand spikes and a naive algorithm prices at the cap — charging desperate people the most at their worst moment. It is defensible in narrow economic terms and **indefensible in every other terms**, and it has repeatedly caused regulatory intervention. The design requirement: automatic emergency suppression, triggered by signals you define in advance (demand anomaly plus external feeds), not by someone noticing the news. **Models optimise the objective you write down. If the objective says nothing about human dignity in a crisis, the model will not supply it.**

! **Fairness and disparate impact**

Persistent surge in lower-income areas with thin driver supply means those riders systematically pay more for the same service. The algorithm is not discriminating on protected attributes — it is faithfully reflecting a supply shortage — but the **outcome** correlates with income and, in many cities, with geography that correlates with caste, religion or race. **Monitor multiplier distribution by area demographics as a standing fairness metric**, and be prepared to subsidise structurally underserved areas rather than letting price fully clear the market.

Regulatory caps change the optimal design

Under an enforced $1.5\times$ cap, price can no longer clear the market at peak — excess demand must be rationed some other way. The design must shift weight toward non-price mechanisms: queueing, an explicit wait-time estimate, demand shaping ("try again in 10 minutes"), driver incentives paid *out of platform margin* rather than rider price, and honest unavailability. **A capped market is a different system design, not the same design with a clip applied** — and candidates who simply add `min(m, 1.5)` have missed the question.

The trade-off table you should be able to draw

No surge: simple, popular, fair-seeming — but long waits, unmet demand, drivers stay home at peak. **Aggressive surge:** market clears, drivers earn, supply responds — but rider anger, regulatory risk, and equity problems. **Capped surge:** the common real-world compromise; needs explicit rationing machinery. There is no correct answer, and saying so *with the trade-offs named* is a stronger interview answer than picking one confidently.

This bonus excerpt stops here. Deep-dives 4 through 19, namely Incentives and Promotions, Location Intelligence, Traffic and Map Inference, ETA Prediction, Dispatch and Matching, Ride Pooling, Driver Positioning, Driver Lifecycle, Payment Fraud, In-Trip Safety, Identity and Documents, Super-App Discovery, Support and Disputes, Multilingual and Voice, Rider Lifecycle, and the Experimentation Platform, are not included in this copy, and this material is not part of the course. **Use the ML Map and the ten-step rubric you just saw worked through three times to design the remaining systems on your own.** That repetition, not more reading, is what builds the skill.